1. Aufgabe: Versionierung/Versionsverwaltung (5 Punkte)

→ Ein Merge (Dateiverschmelzung) ist das Integrieren von bereits im Repository vorhandenen Änderungen in die eigene, lokale Datei. Dieser Schritt ist beim Einchecken notwendig, wenn eine Datei von mehreren Entwicklern gleichzeitig ausgecheckt und in der Zwischenzeit bereits von einem anderen Entwickler aktualisiert wurde. Die Zusammenführung dieser beiden Dateiversionen kann dabei entweder völlig konfliktfrei erfolgen oder zu einem Konflikt führen.

2. Aufgabe: Design Patterns (40 Punkte)

a)

BahnhoefeModel:

```
public class BahnhoefeModel {
    private Bahnhof bahnhof;

    public BahnhoefeModel(){
        bahnhof = new Bahnhof("Berlin Hauptbahnhof", 30, 2019);
    }

    public Bahnhof getBahnhof() {
        return bahnhof;
    }

    public void setBahnhof(Bahnhof bahnhof) {
        this.bahnhof = bahnhof;
    }
}
```

BahnhoefeControl:

```
public class BahnhoefeControl {
    private BahnhoefeView view;
    private BahnhoefeModel model;

    public BahnhoefeControl(Stage primaryStage){
        this.model = new BahnhoefeModel();
        this.view = new BahnhoefeView(primaryStage, model, this);
    }
```

BahnhoefeView:

```
public class BahnhoefeView {

    private GridPane grid = new GridPane();
    private TextField txtBahnhoefe = new TextField();
    private Button btnBahnhoefe = new Button("Bahnhoefe lesen");

    private BahnhoefeControl control;
    private BahnhoefeModel model;

    public BahnhoefeView(
            BahnhoefeControl control,
            Stage primaryStage,
            BahnhoefeModel model
    ){
        this.model = model;
        this.control = control;

         this.grid.setAlignment(Pos.CENTER);
         this.grid.setHgap(15);
         Scene scene = new Scene(grid, 500, 150);
         this.primaryStage.setScene(scene);
         this.primaryStage.show();
         this.initKomponenten();
         this.initListener();
    }

        ...
```

b)

```
public class BahnhoefeModel {
        ...

    public void leseBahnhofAusTxt() throws IOException {
        BufferedReader ein = new BufferedReader(new
FileReader("Text.txt"));
        this.bahnhof = new Bahnhof(
                 ein.readLine(),
                Integer.parseInt(ein.readLine()),
                Integer.parseInt(ein.readLine())
        );
    }

        ...
}


public class BahnhoefeControl {
        ...

    public void leseBahnhofAusTxt(){
        try {
            model.leseBahnhofAusTxt();
        } catch (IOException e) {
            this.view.zeigeFehlermeldungAn("Datei konnte nicht gelesen
                                                werden");
        }
    }

        ...
}


public class BahnhoefeView {
        ...

    private void initListener(){
        btnBahnhoefe.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                control.leseBahnhofAusTxt();
                setTxtBahnhoefe();
            }
        });
    }

    void setTxtBahnhoefe(){
        this.txtBahnhoefe.setText(model.getBahnhof().gibAttributeZurueck());
    }
        ...
}
```

3. Aufgabe: Vertiefung Java (32 Punkte)

a)

```
public class Anwendersystem {
    public static void main() {
        Bahnhof bahnhofBo = new Bahnhof("Bochum Hauptbahnhof", 7, 2010);
        BahnhofBahnhofsgebaeude bahnhofEs = new
BahnhofBahnhofsgebaeude("Essen Hauptbahnhof", 15, 2012, true);

        ArrayList<Bahnhof> liste = new ArrayList<>();
        liste.add(bahnhofBo);
        liste.add(bahnhofEs);

        for (Bahnhof bahnhof : liste) {
            System.out.println(bahnhof.gibAttributeZurueck());
        }
    }
}
```

b)

```
public class BahnhofUtil <T extends Bahnhof>{
    private T bahnhofBasis;

    public BahnhofUtil(T bahnhofBasis){
        this.bahnhofBasis = bahnhofBasis;
    }

    public void gibBahnhofBasisAus(){
        System.out.println("Attribute: " +
bahnhofBasis.gibAttributeZurueck());
    }
}
```

```
public class Anwendersystem {
    public static void main() {
        ...

      BahnhofUtil<Bahnhof> bahnhofUtilBo = new BahnhofUtil<>(bahnhofBo);
      BahnhofUtil<BahnhofBahnhofsgebaeude> bahnhofUtilEs = new
      BahnhofUtil<>(bahnhofEs);

      bahnhofUtilBo.gibBahnhofBasisAus();
      bahnhofUtilEs.gibBahnhofBasisAus();

      ...
    }
}
```

c)

```
public class Anwendersystem {
    public static void main() {
        ...

      Supplier<String> stringSupplier = () -> "Hat der Bahnhof " +
      bahnhofEs.getName() + " ein Gebaeude? ";
      BooleanSupplier booleanSupplier = () ->
      bahnhofEs.isMitBahnhofsgebaeude();

      System.out.println(stringSupplier.get() +
      booleanSupplier.getAsBoolean());

      ...
    }
}
```

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (23 Punkte)

JUnit 5

```
public class BahnhofTest {
    private Bahnhof bahnhof;

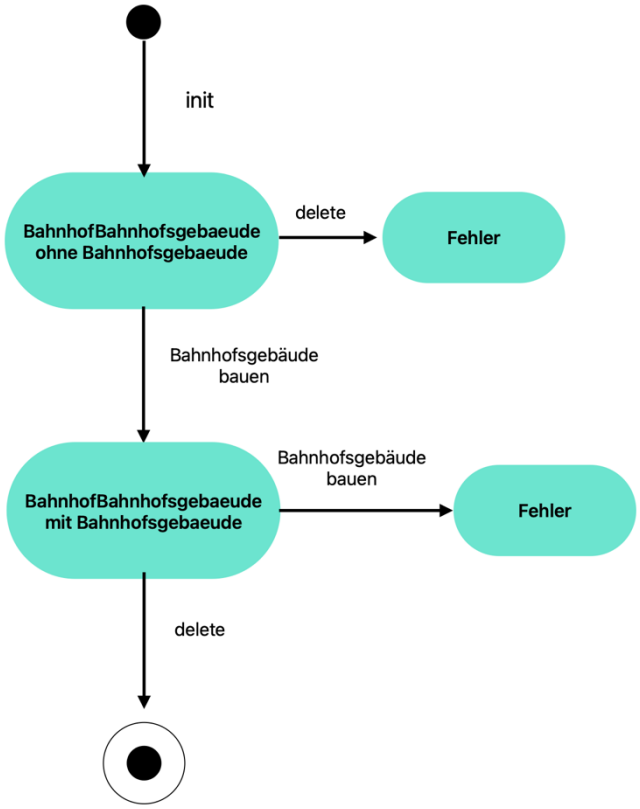
    @BeforeEach
    public void setUp() throws Exception{
        this.bahnhof = new Bahnhof("Berlin Hauptbahnhof", 40, 2019);
    }

    @AfterEach
    public void tearDown() throws Exception{
        this.bahnhof = null;
    }

    @Test
    public void test() throws Exception{
        int actual = 2024;
        bahnhof.setJahrLetzteRenovierung(actual);

        assertTrue(
                () -> this.bahnhof.getJahrLetzteRenovierung() == actual,
                () -> "Das tatsächliche Jahr ist nicht gleich!"
        );
    }
}
```

b)



init / Bahnhofsgebäude bauen / delete
→ Ein BahnhofBahnhofsgebaeude-Objekt, ohne Bahnhofsgebaeude wurde angelegt, mit Bahnhofsgebäude gebaut und dann gelöscht

init / delete
→ Ein BahnhofBahnhofsgebaeude-Objekt, ohne Bahnhofsgebaeude, wurde angelegt und dann gelöscht. Fehlermeldung beim Löschen

5. Aufgabe: SOLID und Clean Code (20 Punkte)

a)

→ Erläuterung:
- Große Schnittstellen sollen in mehrere kleinere, spezifischere Schnittstellen aufgeteilt werden.
- Dies verhindert, dass implementierende Klassen gezwungen werden, Methoden bereitzustellen, die sie gar nicht benötigen

→ Beispiel (Bahnhof / BahnhofBahnhofsgebaeude):
- Hätte man ein einziges großes Interface für alle Bahnhöfe, das die Methode `isMitBahnhofsgebaeude()` verlangt, müsste auch die normale Klasse `Bahnhof` diese Methode sinnloserweise implementieren.
- Nach dem ISP würde man die Schnittstelle aufteilen: Eine für allgemeine Bahnhofsfunktionen und eine separate für gebäudespezifische Funktionen.

b)

- **Entkopplung**: Die Software wird in entkoppelte Klassen aufgeteilt, die sich leichter refaktorisieren lassen.
- **Geringere Abhängigkeiten**: Clients sind nicht mehr von Details eines Services abhängig, die sie gar nicht brauchen (geringere Kopplung).
- **Sicherheit bei Anpassungen**: Zukünftige fachliche oder technische Anforderungen benötigen nur noch punktuelle Änderungen. Das senkt das Risiko von unerwünschten Seiteneffekten und neuen Fehlern enorm.

c)

→ **Genannter Wert**: Korrektheit.
→ **Erläuterung**: Der Code soll frei von Fehlern sein.
→ **Umsetzung im Bahnhofs-Beispiel**:
- Zur Sicherstellung der Fehlerfreiheit wurden in Aufgabe 4 Unit-Tests mit JUnit 5 implementiert (z. B. Test der Methode `setJahrLetzteRenovierung`).
- Außerdem wurde ein zustandsbasierter Test für das Gebäude-Objekt entworfen, um fehlerhafte Zustandsübergänge auszuschließen.

d)

→ Nein, dieses Prinzip wurde im Bahnhofs-Beispiel nicht umgesetzt.
- **Begründung**: Die Klasse `BahnhofBahnhofsgebaeude` erbt über das Schlüsselwort `extends` direkt von der Basisklasse `Bahnhof`.
- Hätte man FCoI angewendet, hätte man auf die Vererbung verzichtet und stattdessen der einfachen Klasse `Bahnhof` ein eigenständiges Objekt/Attribut für das Gebäude hinzugefügt.